C语言与程序设计

The C Programming Language

第10章 文件的输入与输出

毛伏兵

华中科技大学计算机学院

第10章 文件的输入与输出

- 存储在变量和数组中的数据是临时的

  程序结束后都会消失
- 文件用来永久地保存大量数据
- 文件被存储在外存储设备中（如硬盘）
- 本章讨论怎样用C程序建立、更新、处理数据文件

例1、从键盘输入若干行字符，保存到 d:\a.txt 文件中，直到遇到ctrl+z结束.

文件操作的步骤：

（1）打开文件——建立文件指针(或文件描述符)与文件间联系；

（2）通过文件指针对文件进行读写操作；

（3）关闭文件——取消文件指针(或文件描述符)与文件间的联系。

提示：打开文件时，确定了文件读写格式和读写方式！

定义函数实现：将键盘输入存盘到磁盘文件中

- 函数名： copy_term
- 函数参数：

  filename：存盘的文件名， char *
- 函数返回值：无

```
/*  将键盘输入存盘到 filename 文件中  */
void copy_term (const char * filename)
{
   FILE *fp;
   char ch;
   if ( (fp = fopen(filename,"w") ) == NULL){
        printf("can't open the file!");
        exit(-1);
   }
   while((ch = getchar( ) ) != EOF)
        fputc(ch,fp);
   fclose(fp);
}
```

```
#include<stdio.h>
#include<stdlib.h>
void copy_TERM(const char * filename);
int main(void)
{
   copy_TERM ("d:\\a.txt");
   return 0;
}
```

\源程序\ L1.c

打开文件函数fopen()

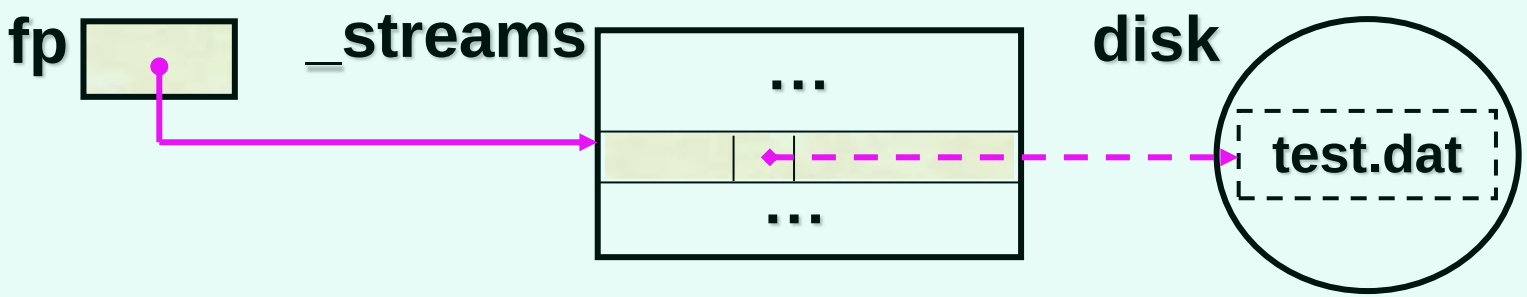
```
FILE * _Cdecl fopen(const char * filename, const char * mode);
```

按照mode方式，打开文件filename。最多可打开OPEN_MAX个文件

成功：返回一个指向filename的文件指针

不成功：返回空指针NULL

```
FILE * fp;
fp＝fopen(“c:\\test.txt”,”w”);
```



打开方式 mode

r —— 只读（read）
w —— 只写（write）
a —— 添加（append）
+ —— 更新
b —— 二进制文件（binary）

这些mode符，可以组合；如果其中不含有“b”，则表示以文本文件方式打开。

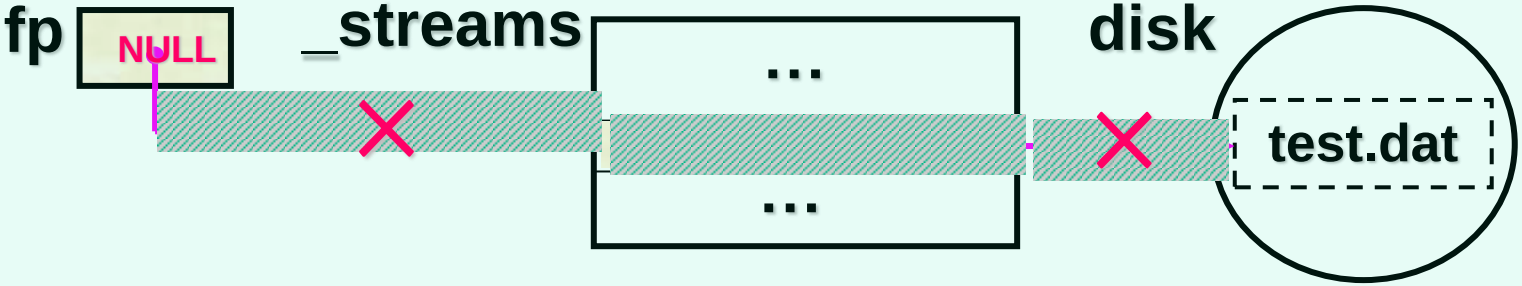
r	以读方式打开文本文件
w	以写方式打开或创建文本文件(清除原内容)
a	以写方式打开或创建文本文件(尾部添加)
rb	以读方式打开二进制文件
wb	以写方式打开或创建二进制文件(清除原内容)
ab	以写方式打开或创建二进制文件(尾部添加)

r+	以读写方式打开文本文件
w+	以读写方式打开或创建文本文件(清除原内容)
a+	以读写方式打开或创建文本文件(尾部添加)
r+b或rb+	以读写方式打开二进制文件
w+b或wb+	以读写方式打开或创建二进制文件(清除原内容)
a+b或ab+	以读写方式打开或创建二进制文件(尾部添加)

关闭文件函数fclose

```
int    _Cdecl  fclose(FILE *stream);
```

关闭文件指针stream“指向”的文件，
正常：返回0
出错：EOF（-1）。



基于字符的文本文件读写 fgetc和fputc

int fgetc(FILE *stream);

从文件指针stream联系的文件中读取一个字符，并将该字符作为unsigned char类型转换为整型值返回。到文件尾或读操作出错时返回EOF。

int fputc(int c, FILE *stream);

参数c转换成为unsigned char类型然后写到文件指针stream指向的输出流的当前位置处。返回值是被写字符；如果写操作出错或遇到文件尾函数返回EOF 。

FILE结构类型(记载文件的全部信息)

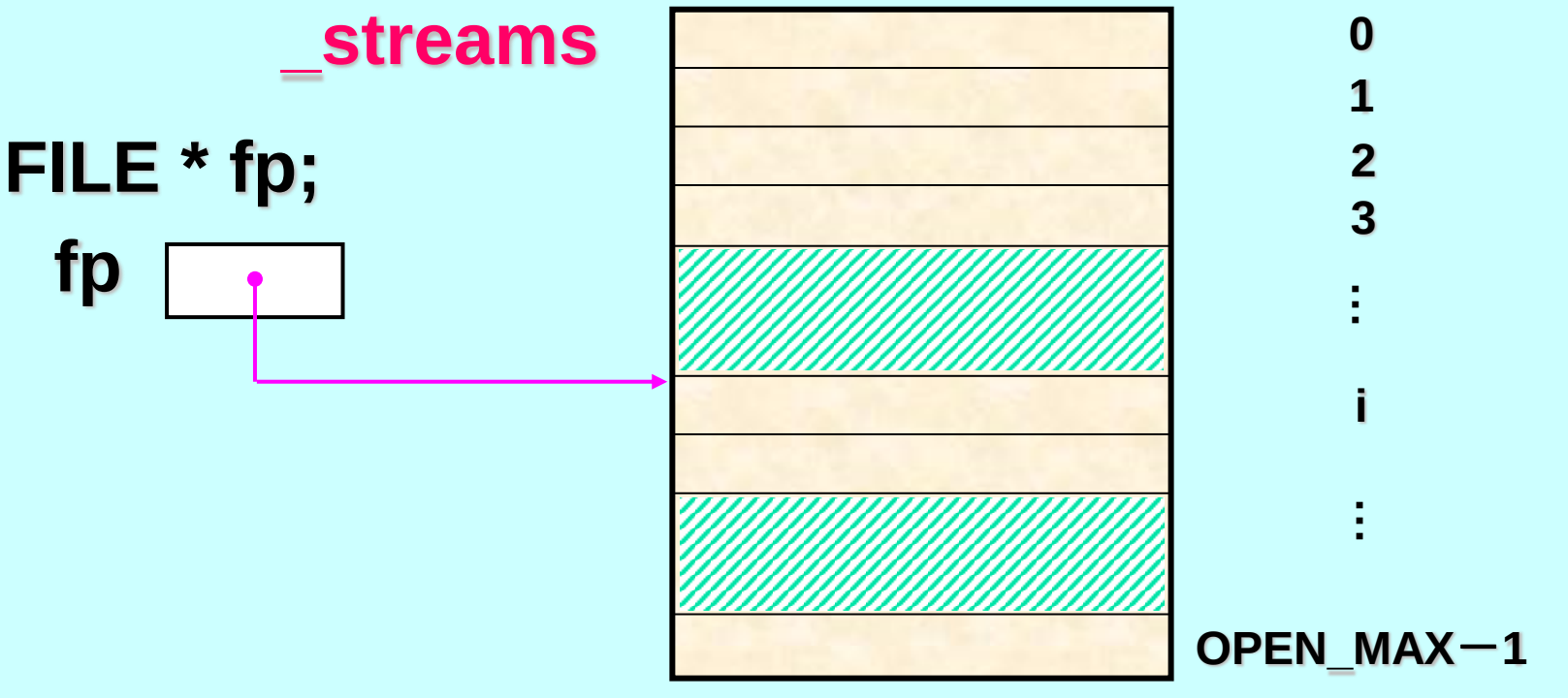
FILE是C语言标准中定义的一种结构类型名称。

```
struct _iobuf {
    char *_ptr;   /*当前读写位置指针，即缓冲区中偏移量*/
    int   _cnt;    /*缓冲区空满*/
    char *_base;     /*指向缓冲区基地址指针*/
    int   _flag;       /*文件状态标志*/
    int   _file;       /*文件描述符*/
    int   _charbuf;   /*非缓冲回送字符*/
    int   _bufsiz;     /*缓冲区大小*/
    char *_tmpfname;/*临时文件描述符*/
 };                  /* 结构类型命名       */
 typedef struct _iobuf FILE;
```

文件描述符是个简单的整数，用以标明每一个被打开的文件。第一个打开的文件是0，第二个是1，依此类推。

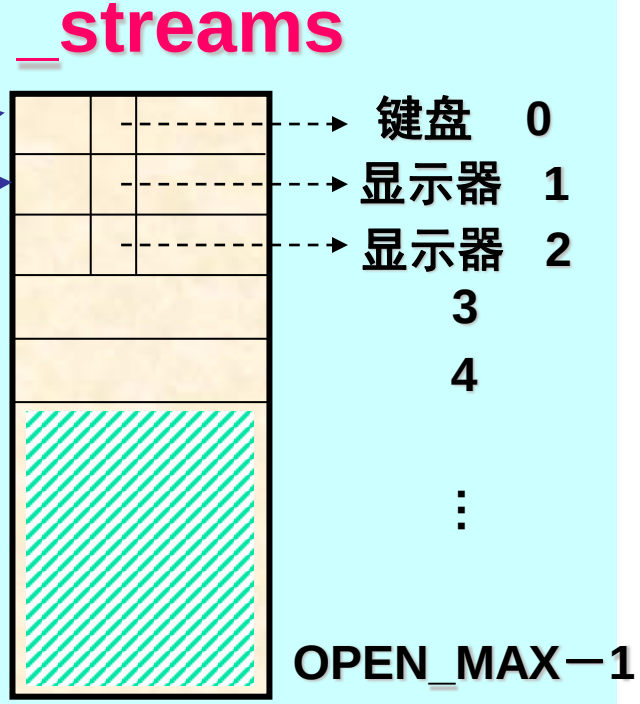
文件指针

程序中仅使用FILE*型变量，通常简称为FILE指针。

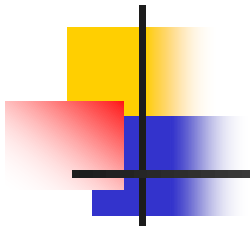


自动打开的三个标准流文件

```
#define stdin  (&_streams[0])
/*标准输入设备，通常是键盘 */
#define stdout  (&_streams[1])
/*标准输出设备，通常是显示器 */
#define stderr  (&_streams[2])
/*标准出错设备----显示器 */
```



stdin称为标准输入文件，stdout称为标准输出文件 。



getchar、gets和scanf函数是从stdin文件读数据；

putchar、puts和printf函数则是向stdout 文件写数据。

freopen函数可以重定向stdin和stdout。如，定向到磁盘文件。

文件的重定向函数freopen()

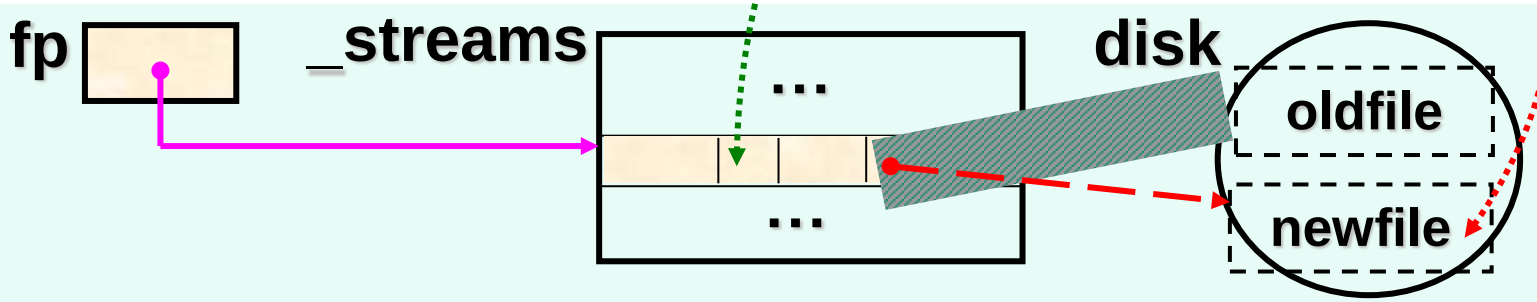
FILE * _Cdecl freopen(const char * filename,const char *mode,FILE *fp);

（1）fclose（fp）；

（2）fp=fopen（filename，mode）；

成功：返回fp

失败：返回NULL。



例2、读入d:\aaa.dat 文件中的内容在屏幕上显示出来

定义函数实现：从磁盘文件中读数据并显示

- 函数名： mytype
- 函数参数：

  filenane：待读数据的文件名， char *
- 函数返回值：无

```
/*  将filename文件中的内容在屏幕上显示出来  */
void mytype(const char * filename)
{
    char ch;
    FILE *fp;
    if((fp = fopen(filename,"r")) == NULL){
        printf("can't open the file!");
        exit(-1);
    }
    while( (ch = fgetc(fp)) != EOF)
        putchar ( ch );
    fclose(fp);
}
```

```
#include<stdio.h>
#include<stdlib.h>
void mytype(const char * filename);
int main(void)
{
    mytype ("d:\\a.dat");
   return 0;
}
```

文件类型

文件按照数据格式分为文本文件和二进制文件两类。

- 文本文件（ASCII文件）

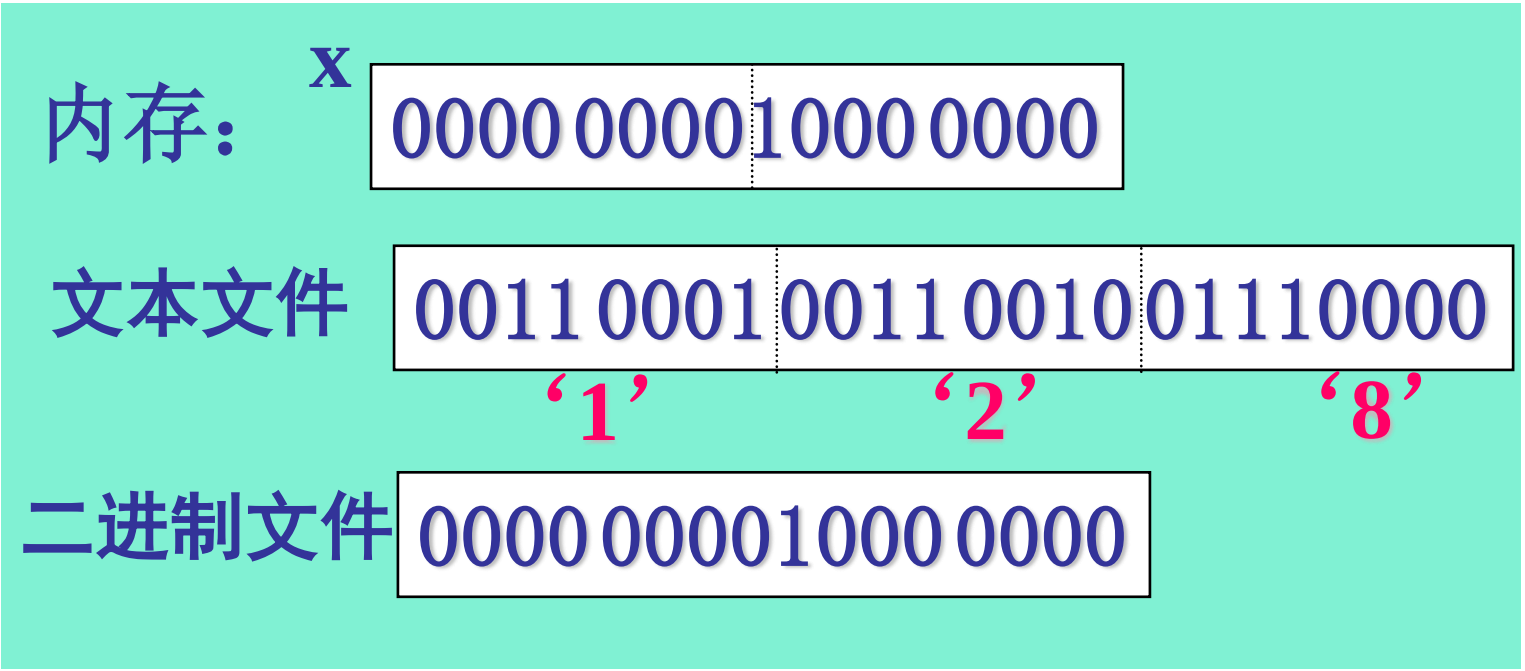
  由字符的ASCII码组成的数据文件（’\n’ -> ‘\r’ 和 ’\n’）。

- 二进制文件

  内存表示的数据序列组成的数据文件

文本文件和二进制文件的存储

短整数x=128以文本文件和二进制文件在磁盘上分别占多少字节？



文本文件的读写

- 基于字符：fputc函数和 fgetc函数
- 基于字符串：fputs函数 和 fgets函数
- 格式化：fprintf 函数和 fscanf函数

getchar和putchar由下面的宏定义。

```
#define getchar( )  getc(stdin)
#define putchar(c) putc((c), stdout)
```

基于字符串的文件读写

调用标准C 提供的字符串输入输出函数每次从输入文件中读或者向输出文件写一个字符串。

```
char * gets(char *s);
int  puts(const char *s);
char * fgets(char *s, int n, FILE *stream);
int  fputs(const char *s, FILE *stream);
```

char * fgets(char *s, int n, FILE *stream);

从stream指向的流中读一行（行长度<n）或至多读n-1个字符（行长度≥n）到s指向的字符数组中。fgets函数正常返回s，出错或遇文件尾返回NULL。

当遇到文件尾且无字符读入s指向的数组时，数组保持原有值不变。

```
char s[10];
fgets（s，10，stdin）；/* 从键盘输入*/
```

int fputs(const char *s, FILE *stream) ;

将指针s 指向的字符串写入stream指向的流中，终结符不写。函数返回一个非负值；如果写操作出错，fputs返回EOF 。

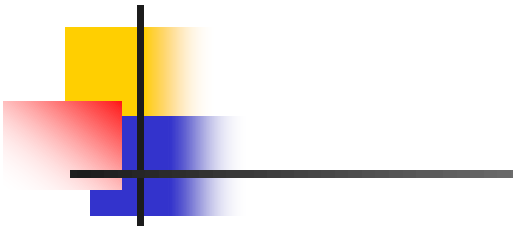
fputs函数与fgets配对使用。

```
fputs（s，stdout）；
```

文件的格式读写

```
int  printf(const char *format, ...);
int  scanf(const char *format, ...);
int  fprintf(FILE *stream, const char *format, ...);
int  fscanf(FILE *stream, const char *format, ...);
int  sprintf(char *buffer, const char *format, ...);
int  sscanf(const char *buffer, const char *format, ...);
```

输出函数正常时都返回输出字符个数，写操作出错时返回负值。输入函数时都返回已输入项数，如果读操作出错都返回EOF。



三对函数的共同特点是它们的功能相同，输入输出都受format指针所指的格式字符串控制， 且使用方法与第3章介绍的printf、scanf用法相同。

三对函数的相异之处在于它们输入输出的对象不同。

教材【例10.6】 文本文件的分解。

- 将一个大的文本文件以行为单位分解成为若干个较小的文本文件，文件名和分解的行数都由用户从命令行输入。

```
/*将文件abc.dat分解为a.dat，b.dat 和c.dat三个文件，分解的行数为10*/
  parts abc.dat a.dat b.dat c.dat 10
```

演示：parts.exe

parts abc.dat a.dat b.dat c.dat 10

（1）len=命令行中分解的行数

atoi（argv[argc-1]）

（2）以读方式打开源文件

fopen（“argv[1]”，“r”）

（3）依次打开目标文件，从源文件读len行写入目标文件（命令行中给出几个源文件？）

for（i=2；i<argc-1;i++){

- 以写方式打开文件argv[i]
- 从argv[1]读1行写入argv[i]直到写了len行或源文件到文件尾。
- 关闭 文件argv[i]

}

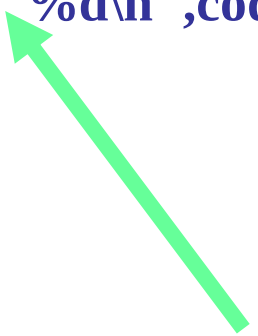
（4）关闭源文件argv[1]

\源程序\ parts.c

教材【例11.9】 编写一个数据采集和处理程序。暂定数据从键盘输入手工采集，采集某种商品到货批次（code）和数量（number）。

```
#include<stdio.h>
#include <stdlib.h>
#define True   1
void data_write(char *);        /*  数据采集并存盘*/
void data_cal(char * ,float );    /*从文件读入数据并进行计算*/
void main(void)
{
    char a[20] = "data.dat";
    data_write(a);
    data_cal(a,  31.4);     /* 31.4是单价 */
}
```

```
/*data_write：从键盘采集某种商品到货批次和数量，存于filename文件中
   */
void data_write(char * filename)
{
    FILE * out;
    char code[5];
    int number = 1;
    if((out = fopen(filename,"w")) == NULL)
        exit(-1);
    printf("input code and number please!\n");
    while( True ){
        scanf("%s%d",code, &number); /* 输入批次，数量 */
        if(number == 0) /* 如果数量为0，结束输入 */
            break;
        fprintf(out,"%s  %d\n",code,number);
    }
    fclose(out);
}
```



必须有空格，便于能够正确进行读

```
/*data_cal: 从filename文件中读入某种商品到货批次和数量，按单价price计算
    总金额，并显示计算结果 */
void data_cal(char * filename,float price)
{
    FILE * in;
    char code[5];
    int number;
    if((in = fopen(filename,"r")) == NULL)
        exit(-1);
    printf("batch\tamount\tsum \n");
    while( fscanf(in,"%s %d ",code,&number) != EOF {
        /* fscanf函数从in指向的流中读数据到code和number中 */
        printf("%s\t%d\t%8.2f\n",code,number,price*number);
    }
    fclose(in);
}
```

演示\源程序\ ex10-9.c

文本文件数据的间隔符

- 当将内存多个数据写入文本文件的时候，特别注意的是：在两个数据之间增加必需的**间隔符**。
- 目的在于读文本文件时，能够保证获得数据转换的正确性。

```
fprintf(out,"%s   %d\n",code,number);
```

文件的直接输入输出-二进制文件的读写

文件直接输入输出又称为文件成组输入输出。标准C为文件的直接输入输出提供了两个函数fread和fwrite，适用于二进制形式文件的读写。

```
size_t  fread(void *ptr, size_t size, size_t n, FILE *stream);
size_t  fwrite(const void *ptr, size_t size, size_t n,FILE *stream);
```

size_t fread(void *ptr, size_t size, size_t n, FILE *stream);

从stream指向的输入文件中读取至多n个大小为size的记录到指针ptr指向的内存单元中。返回值是实际读取记录数。

文件尾测试函数feof()，如果到文件尾，函数返回非0值，否则返回0。 ferror()测试出错函数，如果出错，ferror函数返回非0值，否则返回0。

size_t fwrite(const void *ptr, size_t size, size_t n,FILE *stream);

从指针ptr指向的内存缓冲区中取n条，大小为size的记录写到stream指向的输出文件中。返回值是写出的记录数， 当返回值小于n时，这种情况只有在写操作出错时出现。

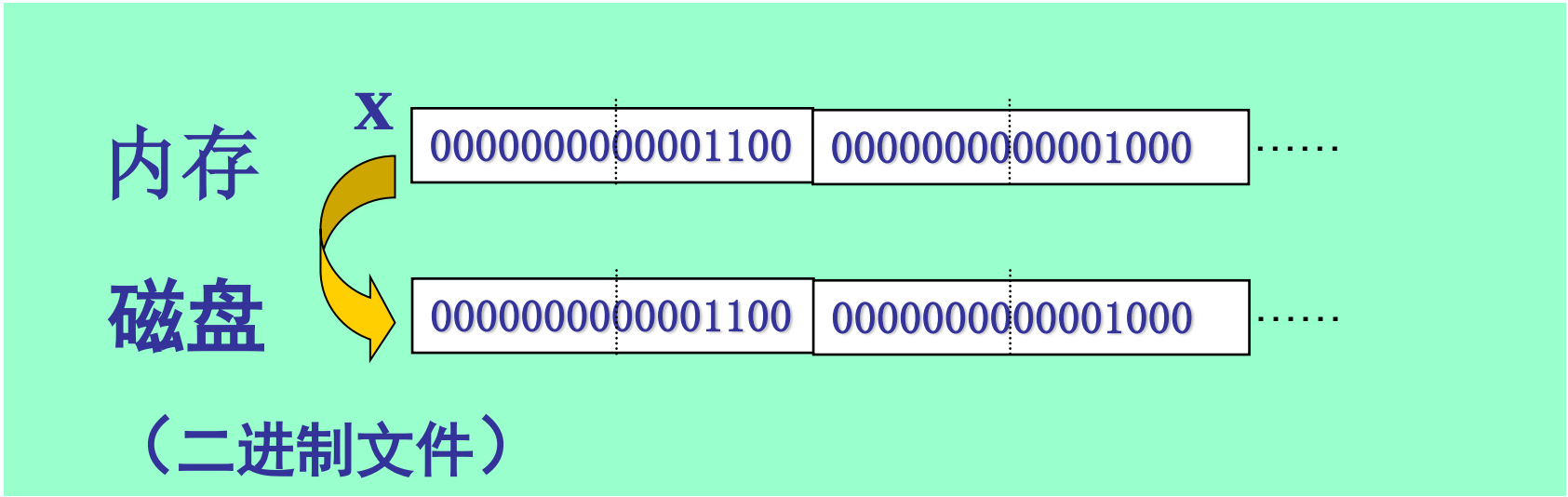
参数的含义同fread， 不同的是stream指向的输出文件。

如何将int x[]={12, 8, 34, 421}输出到磁盘（二进制文件）？

(1) fwrite(x, sizeof(int), 4, fp);

(2) for(i=0;i<4;i++)

fwrite(&x[i], sizeof(int), 1, fp);



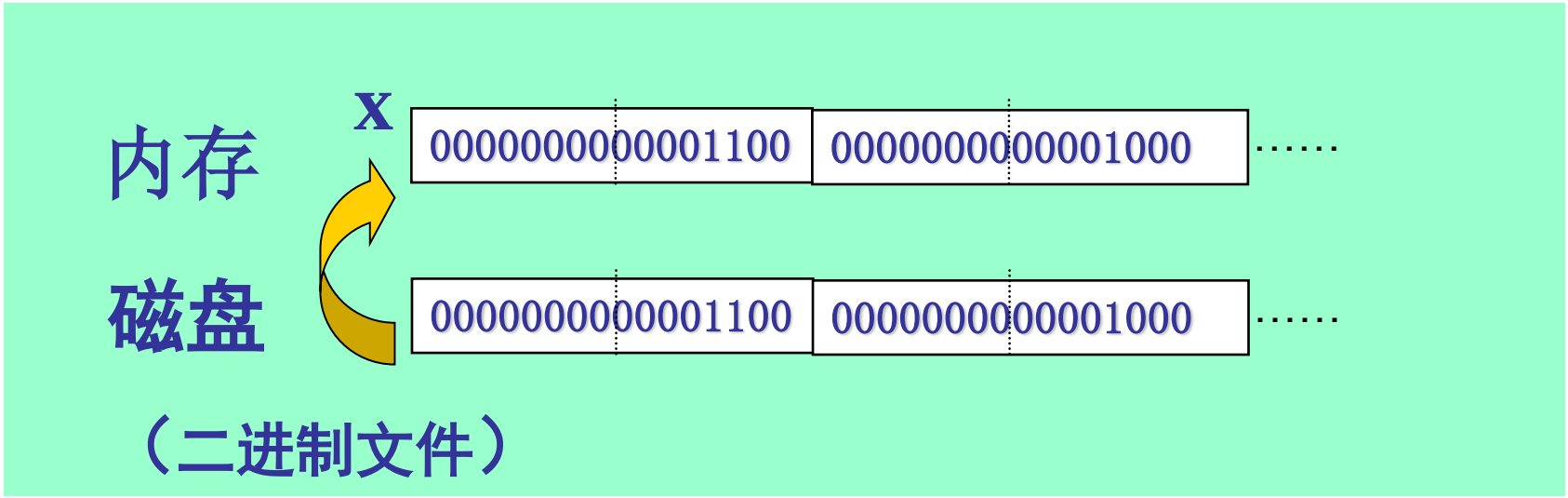
二进制文件数据不需要间隔符

如何从磁盘文件（二进制文件）读数据到内存？

```
int  fread(void *ptr, size_t size, size_t n,
           FILE *stream);
```

——— 二进制文件

```
int i=0, x[10];
while( fread(x+i,sizeof(int), 1,fp) ==1)  i++;
```



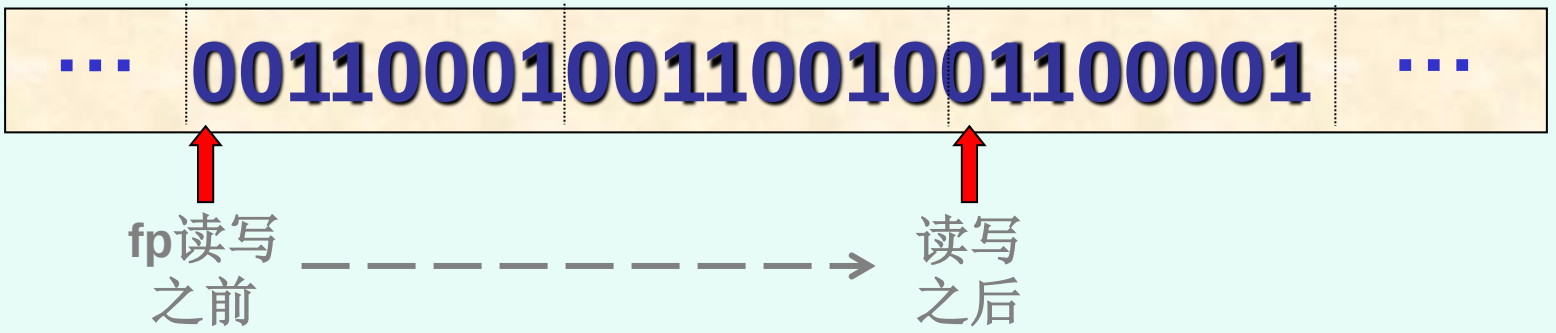
feof()

文件尾测试函数feof()，如果到文件尾，函数返回非0值，否则返回0。 ferror()测试出错函数，如果出错，ferror函数返回非0值，否则返回0。

```
#define  _IOEOF  0x0010

#define  feof(_stream)  ((_stream) ->_flag & _IOEOF)
```

二进制文件数据不需要间隔符



按二进制格式，读入1个int数据：

```
int x;
fread(&x, sizeof(int), 1, fp);
```

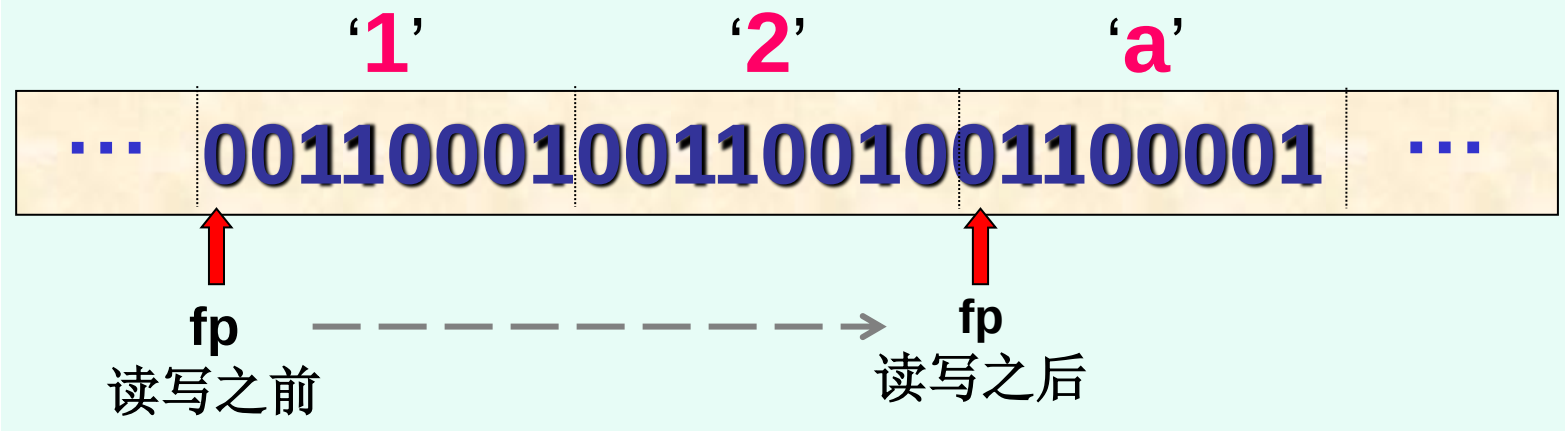
X:12594

文件的读写方式

C语言中，对文件的读写方式有两种：**顺序读写**和**随机读写**。

文件的**顺序读写**：**打开文件**时，读写指针指向**文件头**；读写一个“数据”后，读写指针自动指向下一个“数据”；**程序不用修改读写指针**。数据是从文件头到文件尾，顺序读写。

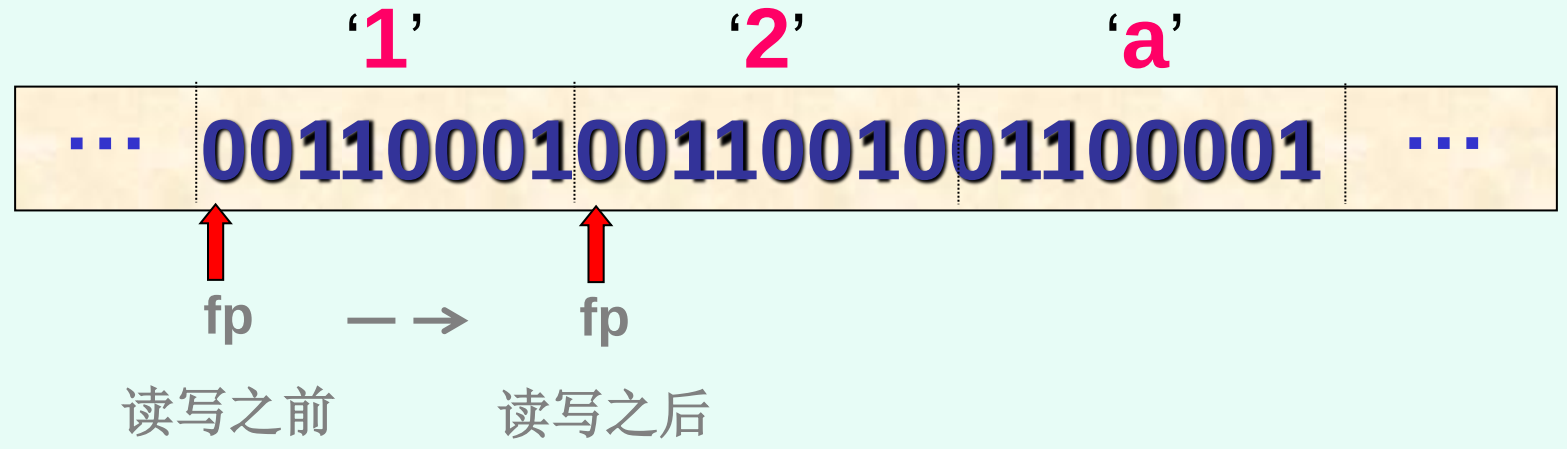
顺序读写



按文本格式，读入1个int数据：

fscanf(fp,”%d”,&x);

x:12， 内存中为：00000000 00001100



按文本格式，读入1个char数据：

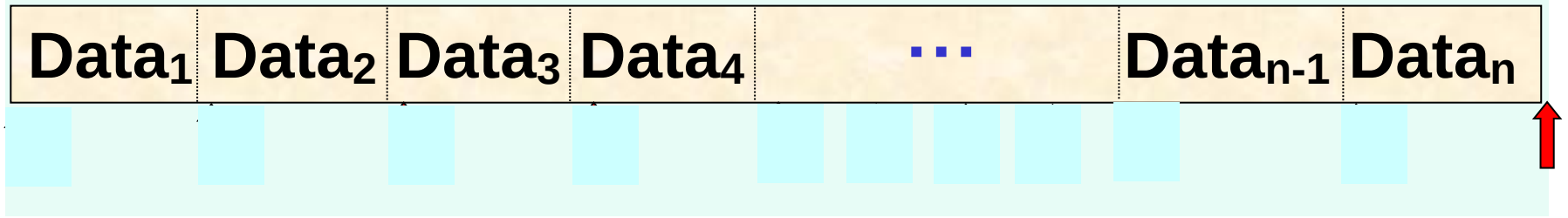
fscanf(fp,”%c”,&x);

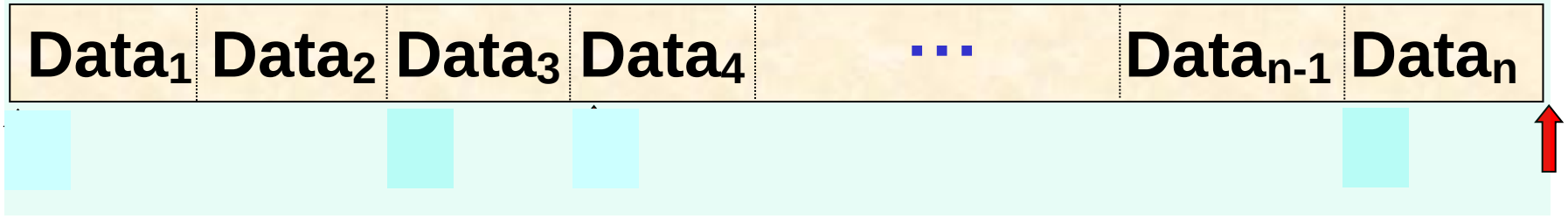
X: ‘1’，内存中为： 00110001

文本文件只能顺序读写
二进制文件能顺序读写，也能随机读写

文本文件（用fprintf建立）数据不一定具有同样的长度

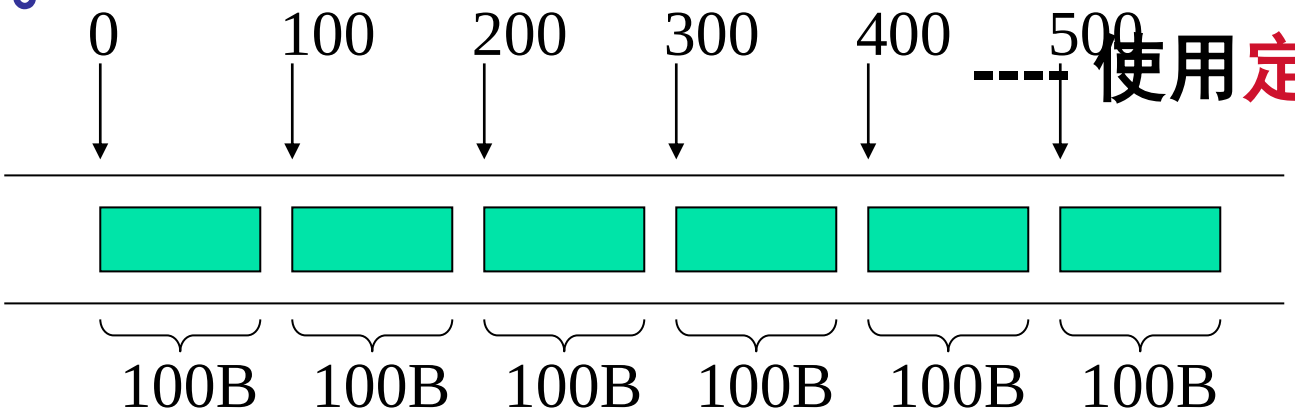
二进制文件（用fwrite建立）数据具有同样的长度





随机读写

随机读写方式：打开文件时，读写指针指向文件头；读写一个“数据”后，读写指针自动指向下一个“数据”；程序能改变读写指针。数据不必从文件头到文件尾顺序读写。



具有定长记录的随机存取文件

流式文件的随机输入输出

1. 文件定位函数

流式文件的随机输入输出是指对文件的读写可以从文件内指定的位置进行，而不必每次从头顺序开始。

```
int fseek(FILE *stream, long offset, int origin);
long ftell(FILE *stream);
int fgetpos(FILE *stream, fpos_t *pos);
int fsetpos(FILE *stream, const fpos_t *pos);
void rewind(FILE *stream);
```

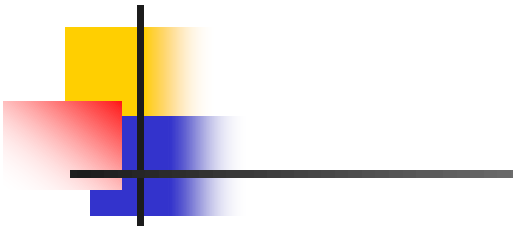
fseek

int fseek(FILE *stream, long offset, int origin);

将stream指向的文件之当前读写指针，定位到origin+offset。如果正常定位，fseek函数返回0，否则返回非0值。 。

origin和offset分别称为基准点和偏移量。





基准点有三种取值：SEEK_SET、SEEK_CUR和SEEK_END

#define SEEK_SET 0 文件起始位置为基准点

#define SEEK_CUR 1 文件当前位置为基准点

#define SEEK_END 2 文件尾部位置为基准点

C标准规定：对二进制文件，基准点的取值与偏移量无关；对文本文件基准点只能取0，而偏移量可以取0或ftell函数的返回值。

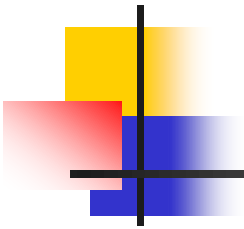


long ftell(FILE *stream);

读取文件读写指针的当前值返回，出错时返回-1L。

int fgetpos(FILE *stream, fpos_t *pos);

将stream指向文件的读写指针当前值，保存到 pos指针所指的 fpos_t类型的对象中。成功保存，fgetpos返回0，否则返回非0值。



int fsetpos(FILE *stream, const fpos_t *pos);

用fgetpos函数保存到pos指针所指对象中的值来设置stream所指向文件的当前位置。设置成功函数返回0，否则返回非0值。

void rewind(FILE *stream);

将文件指针stream指向文件的读写指针重新定位到文件的起始位置，同时清除文件结束标志和出错标志。

2. 文件随机读写

利用文件的定位函数和文件的读写函数，即可实现文件的随机读写。

显然，文件中的数据应该按照某种规律进行组织存储，才便于程序按照这种规律对文件进行读写指针的定位。

定位函数中参数值，依赖于具体的编译程序之约定。在软件开发时，需要认真阅读所使用的开发工具的相关技术资料！

【例10.14】用随机读写方式编制填写班级成绩单文件的程序。

- 假设某班级的学生参加了数学（Math）、物理（Physics）、英语（English）C语言程序设计（C）四门课程的考试，学生试卷按交卷顺序随机排列，然后在试卷不按照学号排序的方式下进行阅卷、登分。
- 分析：由于学生交卷先后顺序有随机性，且各门课程的登分先后也会有随机性，因此比较合理的方式是根据班级学生人数按照学号顺序建立一张空的成绩单文件，然后根据试卷登分时的情况选用单科成绩登分或四科成绩同时登分。随机定位的依据是试卷上学生的学号。
- 源程序\ex10_14.c

例10.18 （使用底层接口函数）读取指定汉字在16×16点阵汉字库中的点阵字模信息，并将汉字点阵字模信息以文本方式显示。

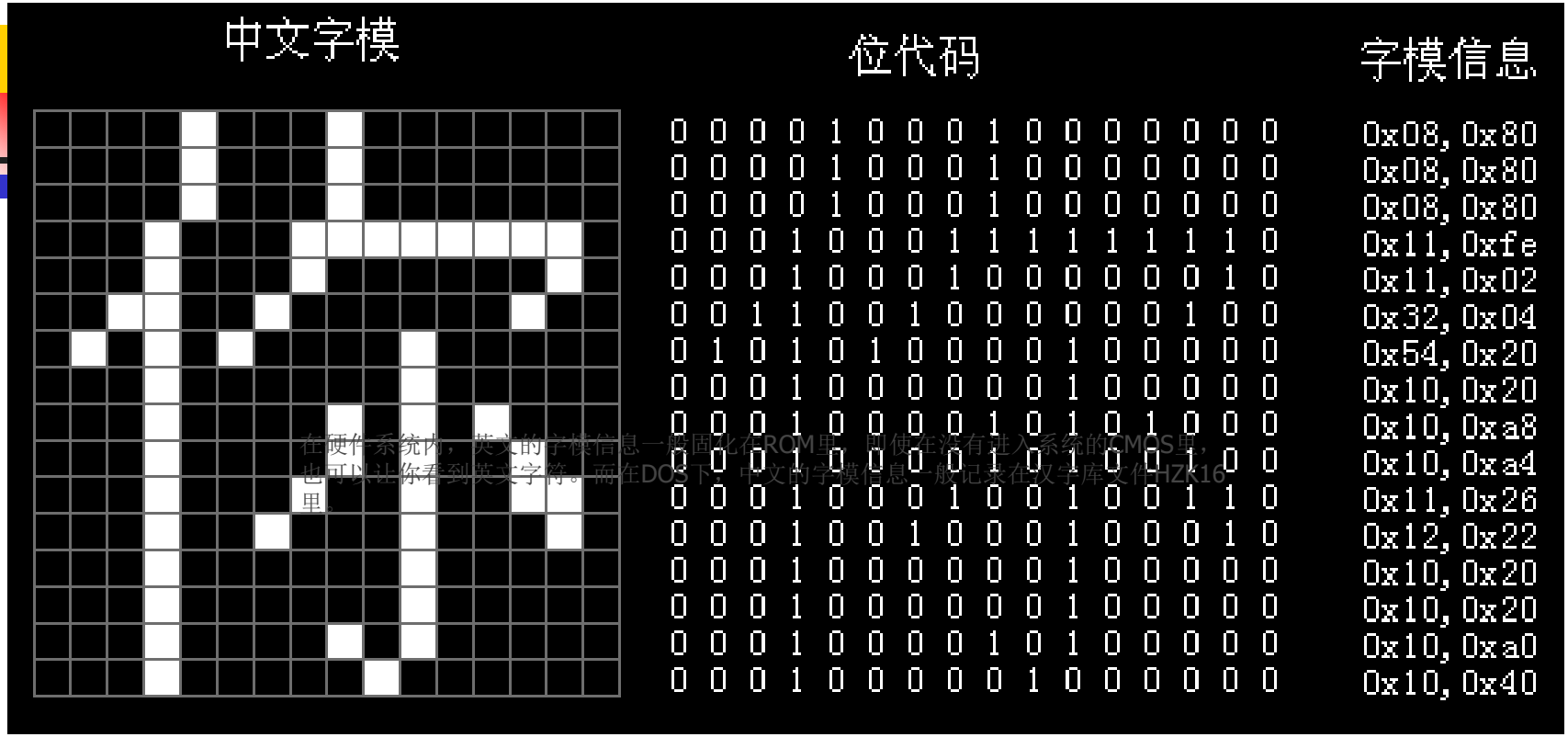
汉字的内码：

中文或西文信息在计算机系统中的代码表示称为机内码。ASCII码是一种西文机内码（1B）。汉字机内码用连续2B表示，每个字节的最高位是1。

机内码 = 国标码+8080H =区位码$_{16}$+A0A0H

- 国标码（2121H～7E7EH）
- 区位码（0101～9494）
- 机内码（A1A1H～FEFEH）
- 汉字内码仅是一组数字，又如何在屏幕上去显示呢？这就涉及到文字的字模，字模也是一组数字，但它的意义却与数字的意义有了根本的变化，它是用数字的各位信息来记载英文或汉字的形状.

汉字字模



在硬件系统内，英文的字模信息一般固化在**ROM**里。而在**DOS**下，中文的字模信息一般记录在汉字库文件**HZK16**里。如何得到汉字的字模信息。

汉字库文件（HZK16）

- HZK16是16*16。还有HZK24，HZK32和HZK48字库及黑体、楷体和隶书字库。它们都是按照区位的顺序排列 的。
- 汉字在汉字库中的具体位置（序号：从0开始）

  index=94*(区号-1)+位号-1。
- 汉字点阵字模在汉字库中的位置（B）：

  index*一个汉字字模占用字节数

源程序\ex10_18_stream.c

其它文件操作函数(自学）

1. 文件访问函数

C标准规定了五个文件操作函数如下，其中前两个已经讨论过。

```
FILE * _Cdecl fopen(const char * filename, const char * mode);
int    _Cdecl  fclose(FILE *stream);
int   fflush(FILE *stream);
int   setvbuf(FILE *stream, char *buf, int mode, size_t size);
void  setbuf(FILE *stream, char *buf);
```

11.5 其它文件操作函数

int fflush(FILE *stream);

将stream指向的输出缓冲区或更新缓冲区（用追加方式打开）中尚未存盘的数据强制性的全部写到介质文件中。如果stream为NULL，执行fflush操作将使所有缓冲区的数据存盘。写操作成功，fflush函数返回0，否则置出错标志并返回EOF。

11.5 其它文件操作函数

int setvbuf(FILE *stream, char *buf, int mode, size_t size);

在打开文件之后，其他任何有关文件的操作被执行之前调用；setvbuf函数用buf指向的存储区作为stream指向流的缓冲，缓冲方式由mode参数决定，缓冲大小由size决定。调用成功，函数返回0，否则返回非0值。

标准C为mode规定了三种模式：_IOFBF(全缓冲)、_IOLBF(行缓冲)和_IONBF(无缓冲)。

11.5 其它文件操作函数

void setbuf(FILE *stream, char *buf);

功能与 setvbuf函数完全一样。不同的是该函数采用全缓冲(_IOFBF)，缓冲大小用BUFSIZ，函数没有返回值。

缓冲大小值BUFSIZ，在stdio.h定义，一般为512个字节。

11.5 其它文件操作函数

2. 文件操作函数

int remove(const char * filename);

int rename(const char * oldname, const char * newname);

FILE * tmpfile(void);

char * tmpnam(char *s);

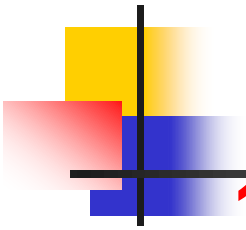
11.5 其它文件操作函数

int remove(const char * filename);

删除以filename指向的字符串为文件名的文件。如果操作成功，remove函数返回0，否则返回非0值。

如果对一个已经打开的文件进行remove操作，实际结果由各个C编译器自行决定。

实际上，remove函数就是DOS的del操作之功能。



11.5 其它文件操作函数

int rename(const char * oldname, const char * newname);

以oldname指向的字符串为文件名，更换成以newname指向的字符串为文件名。如果操作成功，函数返回0，否则返回非0值。

实际上，rename函数就是DOS的ren操作之功能。

11.5 其它文件操作函数

FILE * tmpfile(void);

以"wb+"方式创建一个临时二进制文件，该临时文件在文件关闭或程序结束时被自动清除。如果调用成功，tmpfile函数返回一个指向它所创建的临时文件的文件指针，否则返回NULL。

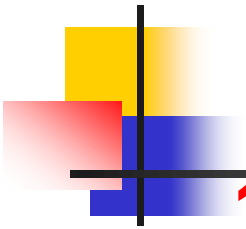
char * tmpnam(char *s);

产生一个与系统中已存在的文件不相同的文件名。当参数非空，函数将产生的文件名存入s指针指向的字符数组，并将s的值作为返回值；如果操作失败，返回一个空指针。

11.5 其它文件操作函数

3. 出错检测处理函数

```
void clearerr(FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
void perror(const char *s);
```



11.5 其它文件操作函数

clearerr函数清除stream 所指流的EOF标志和出错标志，无返回值。

feof函数检测stream所指流是否到达尾。如果到达，返回非0值，否则返回0值。

ferror函数检测stream所指流的出错，如果出错，ferror函数返回非0值，否则返回0值。

perror函数将出错信息(全局变量errno的值)写入字符指针s所指的字符数组中，无返回值。

11.6 输入输出的低层接口(io.h)

与流式文件使用标准C 定义的基于缓冲的函数不同，输入输出的低层接口中使用的函数不是由标准C 定义，而是通过操作系统提供的功能调用(也称为接口)实现。

流式文件使用通过FILE结构定义的缓冲，而输入输出的低层接口不使用FILE文件指针，所使用的是整型文件说明符。

对于不同的操作系统，编译器提供的低层接口形式是统一的。差异之处在于对读写方式、文件属性的规定和符号常量的定义有所不同。

11.6 输入输出的低层接口

1. 文件的顺序输入输出

```
int  creat(const char *path, int amode);
int  open(const char *path, int access,
unsigned mode);
int  read(int handle, void *buf, unsigned
len);
int  write(int handle, void *buf, unsigned
len);
int  close(int handle);
```

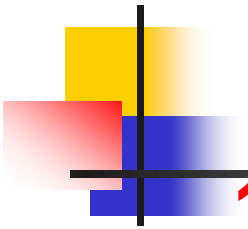
11.6 输入输出的低层接口

2. 文件的随机输入输出

long lseek(int handle, long offset, int origin);

long tell(int handle);

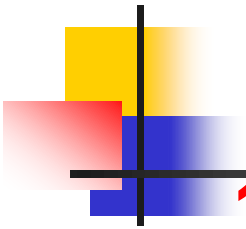
int eof(int handle);



11.7 流式文件系统中库函数的设计

1. fopen函数的设计

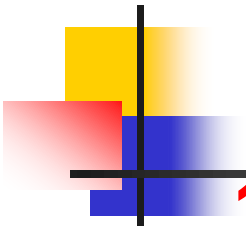
（略）



11.7 流式文件系统中库函数的设计

2. mygetfp函数和my_openfp函数的设计

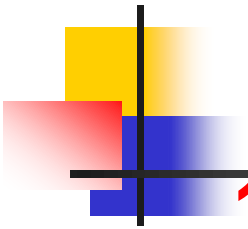
（略）



11.7 流式文件系统中库函数的设计

3. myCheckOpenType函数的设计

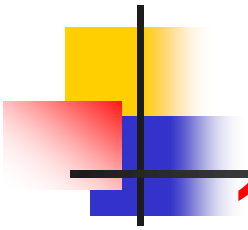
（略）



11.7 流式文件系统中库函数的设计

4. myfread函数的设计

（略）



11.7 流式文件系统中库函数的设计

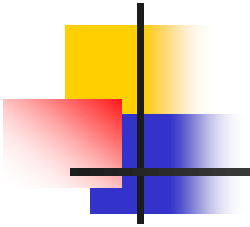
5.myfwrite函数的设计

（略）

11.7 流式文件系统中库函数的设计

6. 使用自己编写的库函数

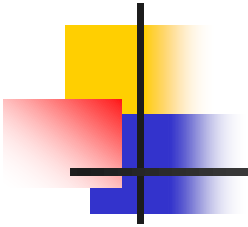
（略）



本章小结

首先介绍了文件的有关概念，包括了文本文件和二进制文件的含义，顺序文件和随机文件的文件的读写方式，以及C程序输入输出的实现方法。

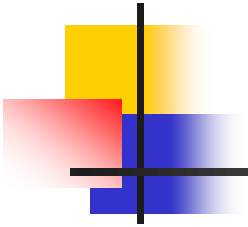
在FILE指针和标准流式文件方面重点介绍了FILE结构类型和FILE指针，标准流式文件，这些概念要熟练掌握。



本章小结

在流式文件的顺序输入输出方面介绍了文件的打开与关闭，文件的重定向，基于字符和基于字符串的文件读写，以及文件的格式读写和文件的直接输入输出。

在流式文件的随机输入输出方面，介绍了文件定位函数，并以随机读写方式编写填写班级成绩单文件的程序介绍了流式文件的随机读写。



本章小结

其次，对C标准规定的其它文件操作函数，如文件访问函数、文件操作函数、出错检测处理函数进行了介绍。

介绍了输入输出的低层接口中的文件顺序输入输出和文件的随机输入输出。

最后，介绍了在流式文件系统中，库函数的设计方法。